

Yesterday you built the machine. Today your code moves in.

Two tools stand between the project on your laptop and the server you launched yesterday: Git, which moves code around, and Docker, which packages it so it runs the same everywhere. By tonight your badge is live on the internet – and deploying itself.

IN PARTNERSHIP WITH



From your laptop to a public address, in four moves

MOVE 1

Confirm the project

Already forked, cloned, and made yours yesterday – a quick check, then move on.

Git

GitHub

VS Code

MOVE 2

Run someone else's

Install Docker on the server and drive it with six commands.

docker run

-p

exec

MOVE 3

Package your own

Write the Dockerfile, build the image, push it, run it on the server.

Dockerfile

Docker Hub

Caddy

MOVE 4

Stop doing it by hand

A robot rebuilds and redeploys on every push to main.

GitHub Actions

CI/CD

Moves 1 and 2 are this morning. Moves 3 and 4 are this afternoon – and move 4 is the one you will still be using in ten years.

Git is save-points for your code, plus a shared home online

Git tracks every change you make and syncs it between computers through a site like GitHub. It matters here for one blunt reason: it is how code gets onto your server – and from this afternoon, what triggers a deployment.

clone	Download a project from GitHub to your computer
add	Mark which changed files you want to save
commit	Save those changes as a labelled snapshot
push	Upload your commits to GitHub

Four actions is enough

That is the whole of Git for today. Branches, pull requests and merges come on Day 5, when you have something worth branching.

You will click, not type

All four happen in VS Code's Source Control panel – no commands to memorise, no SSH keys to configure this morning.

✓ **Commit** is add + commit in one click. **Sync Changes** is push.

The Cloud Explorer Badge – the app you already made yours

A small web page with your name on it

Your colours, a fun fact, your favourite film or game, and a photo booth that uses the camera. A real React app you forked, cloned and personalised yesterday.

You already changed one block

Everything personal lives in a single `CONFIG` object at the top of `src/App.tsx` – the one you edited yesterday.

Need it running locally again? `npm install`, then `npm run dev` → **localhost:5173**.

```
const CONFIG = {
  gradientColors: ["#4F46E5", "#8B5CF6"],
  name: "Your Name",
  funFact: "I own 7 cats.",
  isOnCloud: false, // flip when live
  enablePhotoBooth: true,
  favorite: {
    category: "movie",
    title: "Interstellar",
  },
};
```

Fork first, then clone – why yesterday's choice matters today

Clone the starter directly THE TRAP

Your copy stays pointed at our repository. You can commit locally all day, but the first time you push, GitHub refuses – you have no write access to someone else's repo.

You find out at the worst possible moment: **at the end**.

Fork, then clone your fork WHAT YOU DID

A fork is a full copy under your own account. Clone that, and every commit and push this week goes to a repo you own – including the one that triggers your deployments this afternoon.

On GitHub: **Fork** → **Create fork**, then copy **your** URL.

Already forked and cloned yesterday? You're set – move to the next slide. Haven't yet? In VS Code: **Ctrl/Cmd + Shift + P → Git: Clone** → paste your fork's URL – the one with your username in it, not ours.

Docker packages your app so it runs the same everywhere

Yesterday VIRTUAL MACHINE

Your ECS instance is a whole computer: its own kernel, its own operating system, a minute to boot.

Today CONTAINER

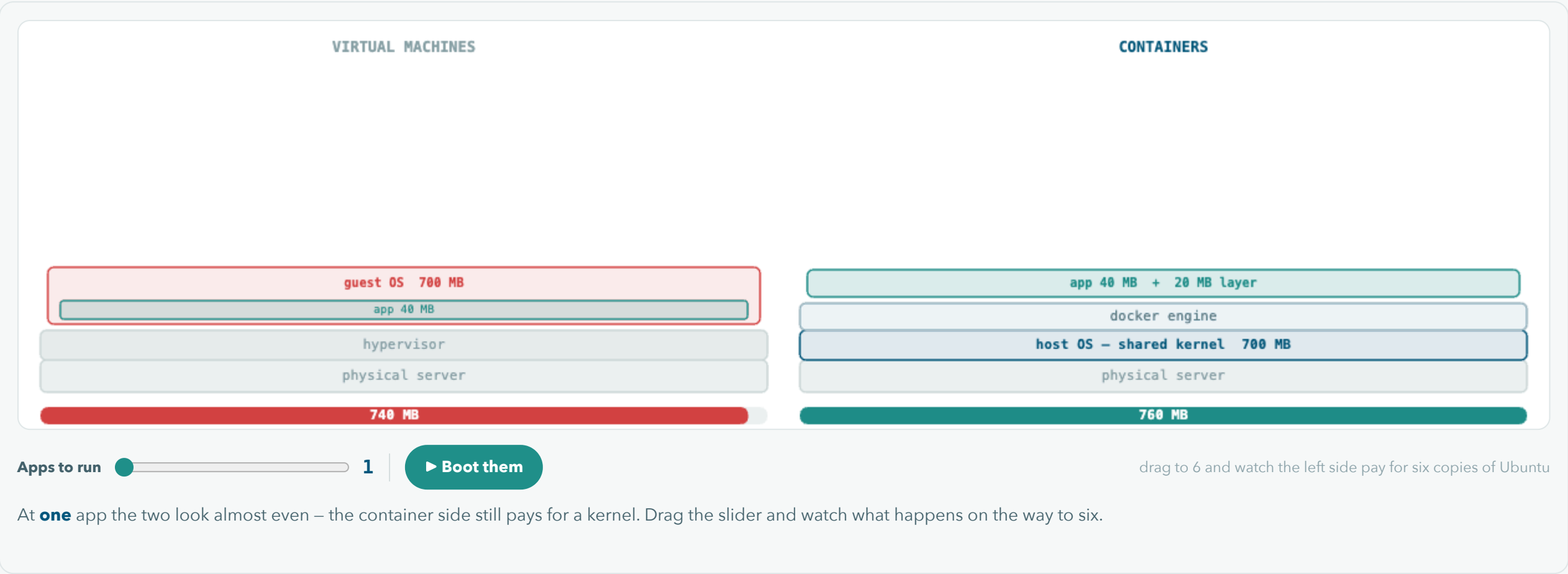
Docker packages just your app and what it needs to run, and shares the host's operating system instead of bringing its own. It starts in seconds and runs the same way on any machine.

Docker THE ENGINE

Docker is the tool that builds, ships and runs containers – the six commands you are about to use all week.

One sentence worth remembering: a container is lighter because it never brings its own operating system. The next slide makes that difference visible.

Watch the VM side buy six copies of Ubuntu



An image is the recipe; a container is the dish you cooked

Image THE RECIPE

A frozen, read-only package of your app and everything it needs. It does not run. You build it once, push it to a registry, and pull it anywhere.

One image → **many** containers, on many machines.

Container THE DISH

A running instance of an image. Start it, stop it, throw it away, start another – the image it came from never changes.

docker run turns a recipe into a dish.

⚠ "Image" now means two different things this week. Yesterday's image was the operating-system template you picked for your ECS instance. Today's image is a packaged application. Same word, different layer.

image:tag, and the registry it came from

Naming IMAGE:TAG

An image's full name is `image:tag`.
The tag picks a version or variant –
leave it off and Docker assumes
`:latest`.

`nginx` really means `nginx:latest`;
`node:22-alpine` names the image and
the exact variant.

Images live in a registry

A registry is a server that stores images
for `docker push` and `docker pull`.
Docker Hub is the default, and the one
this course uses.

Docker Hub isn't the only one

Alibaba Cloud has its own Container
Registry (ACR); GitHub has `ghcr.io`.
Same idea, different host – the image
name just gets a prefix.

`ghcr.io/user/app` instead of
`user/app`.

In a moment, Hisham's task runs `docker run ... nginx`. Docker won't find `nginx:latest` on the server, so it will pull it from Docker Hub automatically, first – before it ever starts the container.

Six commands run every container you meet this week

<code>docker run <image></code>	Start a container from an image
<code>docker run -p 8080:80 ...</code>	Map port 8080 on the host → port 80 in the container
<code>docker run -v /host:/data ...</code>	Mount a host folder – a volume – inside it
<code>docker ps</code>	List running containers
<code>docker exec -it <name> sh</code>	Open a shell inside a running container
<code>docker stop <name></code>	Stop it

-d runs it detached

Without `-d` the container holds your terminal hostage. With it, it runs in the background and you get your prompt back.

exec is your window in

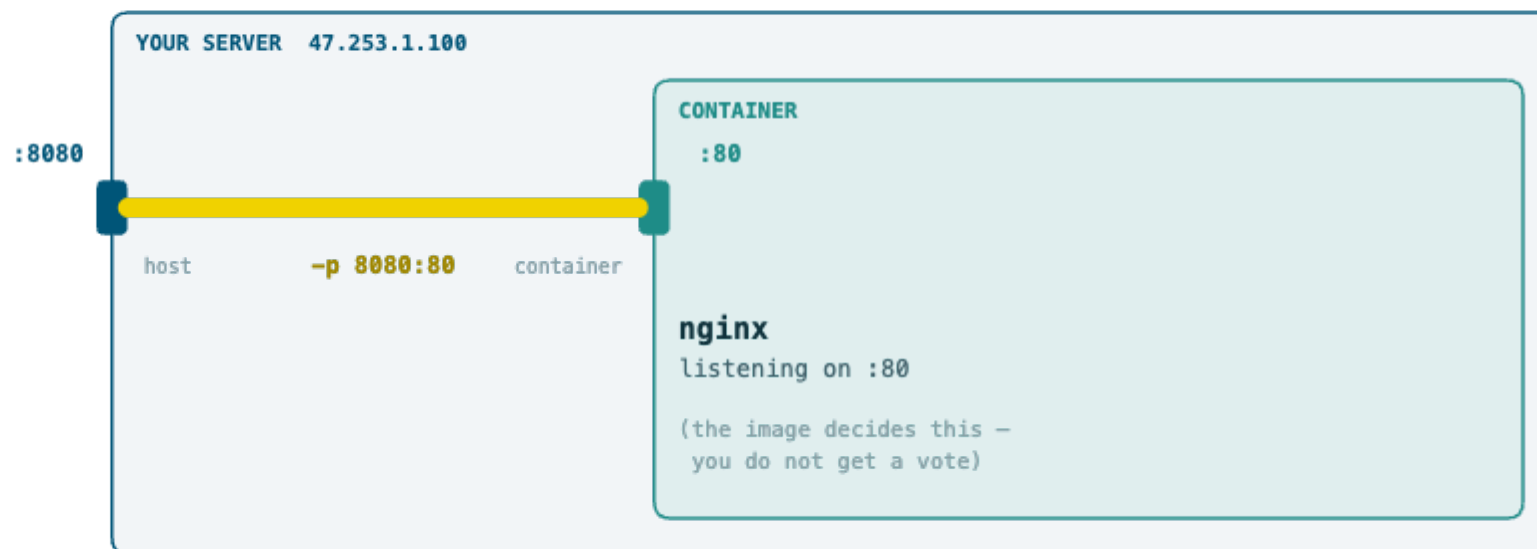
A container is not a black box. `exec -it ... sh` drops you inside with the same `ls` and `cd` you learned yesterday.

Type **exit** to come back out.

Send a request and watch exactly where it dies

```
$ docker run -d -p 8080:80 nginx
```

YOU
curl
:8080



-p

8080

80

:

80

8080

published

► Send a request

now try flipping it to -p 80:8080

-p 8080:80 maps the server's :8080 to the container's :80. Press **Send a request**.

Here is exactly what to run – and why each line matters

Hisham started a container someone handed him and has no idea what is in it. Let's explore the commands.

```
# 1. Run nginx, name it "web", map host 8080 → container 80
docker run -d --name web -p 8080:80 nginx

# 2. List running containers – is "web" there?
docker ps

# 3. Open a shell inside it, then look around
docker exec -it web sh
# (inside) ls /usr/share/nginx/html → then: exit

# 4. Check it serves a page, from the server itself
curl http://localhost:8080

# 5. Clean up
docker stop web && docker rm web
```



Code on GitHub, Docker running on your machine in the cloud

- 1 Your personalised badge is pushed to your own fork.
- 2 SSH into yesterday's server – Terminal on macOS/Linux, Git Bash or MobaXterm on Windows:
`ssh -i ~/my-key.pem root@<your-EIP>`
- 3 Install Docker on it: `curl -fsSL https://get.docker.com | sh`, then confirm with `docker --version`.
- 4 Complete Hisham's task on the server, to prove Docker really works there.

☐ Badge in your own public GitHub repo · `docker --version` works on the server · you ran, `exec`-ed into, and cleaned up the nginx container.

You can push a real project and run containers on your own server

Git, four verbs deep

Fork, clone, commit, push – through VS Code, into a repository you own.

Containers, driven

You ran someone else's image, published a port, opened a shell inside it, and cleaned up after yourself.

After the break

You package your own app into an image, put it live over HTTPS, and then make it deploy itself.

 Kahoot: Day 2 · Session 1 – Git basics, VM vs container, images vs containers, ports, and `exec`.

You can run other people's containers. Now package your own.

This is the session where your badge goes live – a real HTTPS address, a real padlock, a working camera. Then you will feel exactly why nobody deploys by hand for long, and you will fix it before you leave.

Build the site with Node, then serve it with something tiny

Phase 1 – build NODE:22-ALPINE

Install dependencies, compile the React app, produce a folder of finished HTML, CSS and JavaScript in `/app/dist`.

Needs the whole Node toolchain. Needed **once**.

Phase 2 – serve CADDY:2-ALPINE

Take those finished files and serve them over HTTPS. No Node, no compiler, no `node_modules` – a web server and a folder.

This is the **only** phase that ships to your server.

Two phases means **FROM** appears twice in one file. That is a multi-stage build – and the next slide shows what it saves.

Throw the workshop away and post only the parcel

\$ docker build . → two FROMs; only stage 2 ships

STAGE 1 AS build
discarded – never pushed

node:22-alpine	120 MB
node_modules	210 MB
source + config	4 MB
/app/dist	2 MB

STAGE 2 FROM caddy:2-alpine
this is the image you push

caddy:2-alpine	44 MB
/app/dist	2 MB

COPY --from=build

/app/dist
2 MB

ships: 46 MB

single stage

multi stage

COPY --from=build

/app/dist

/app

▶ Build

try copying /app instead – the classic mistake

46 MB. Stage 1 did the work and was **thrown away** – it is never pushed. Stage 2 starts from a tiny web server and **COPY --from=build** carries across only **/app/dist**, the finished site. That is why **FROM** appears twice: the first stage is a workshop, the second is the parcel.

Four blanks, and you have already learned all four

```
# ---- Stage 1: build the static site ----
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN _____ # install dependencies cleanly
COPY . .
RUN npm run build # produces /app/dist

# ---- Stage 2: serve it over HTTPS with Caddy ----
FROM caddy:2-alpine
COPY --from=_____ /app/dist /usr/share/caddy
COPY Caddyfile /etc/caddy/Caddyfile
EXPOSE _____ # the two ports HTTPS needs
```

Hints: a clean install when a `package-lock.json` exists is `npm ci`. The first stage was given a name – look at the **AS**.

The camera will not open on a plain http:// address

The browser blocks it

Camera and microphone are privileged. Browsers hand them over only on a secure origin – HTTPS, or `localhost`. That is why the photo booth worked on your laptop.

Caddy gets the certificate

A tiny web server that fetches a real, trusted certificate automatically and renews it. No purchase, no manual setup – but it must be reachable to prove you own the name.

sslip.io gives you the name

You have an IP, not a domain. 47–253–1–100.`sslip.io` resolves straight back to `47.253.1.100` – free, instant, no DNS to configure.

Your IP, with dots turned into **dashes**.

⚠ Open ports 80 and 443 in your instance's security group now (Alibaba console → your instance → Security Groups → inbound rules). Caddy needs 80 to prove ownership and 443 to serve. Do it now, understand it Thursday.

Your image needs an address of its own

Right now your image exists on exactly one machine: the laptop that built it. Your server cannot see it, and on Day 4 a Kubernetes cluster will not either. It needs to live somewhere both can pull from – a container registry.

Docker Hub is the default one

Free account, public images. Sign in from Docker Desktop and the CLI reuses that login – no `docker login` in the terminal.

Your image becomes `<username>/cloud-explorer`.

```
# on YOUR machine
docker build \
  --platform linux/amd64 \
  -t <user>/cloud-explorer .

docker push <user>/cloud-explorer
```

`--platform linux/amd64` matters on an Apple Silicon Mac: your laptop builds ARM images by default and your server is x86. Skip it and the container refuses to start with an exec-format error.

Pull it on the server, run it, and open the padlock

```
# on YOUR SERVER, over SSH
docker pull <user>/cloud-explorer

docker run -d -p 80:80 -p 443:443 \
  -e SITE_ADDRESS=47-253-1-100.sslip.io \    # your IP, with dashes
  --name cloud-explorer <user>/cloud-explorer
```

Yesterday's environment variable, doing real work

`-e SITE_ADDRESS=...` sets an environment variable inside the container – exactly the concept from Day 1. Caddy reads it to know which hostname to request a certificate for.

Then open it

`https://<ip-with-dashes>.sslip.io` – your badge, a trusted padlock, and a working photo booth. That is your app on the internet. 🌐

Now change one colour and ship it again

You edited `gradientColors`. One line. To get that line onto the internet, by hand, you now do all of this – and then do it again for the next one-line change, and the one after that.

```
commit + push → docker build → docker push → ssh in →  
docker pull → docker rm -f → docker run      # every. single. time.
```

Seven steps, about three minutes of your attention, and one of them is easy to forget at 5pm on a Thursday. Forget the wrong one and the site looks fine while serving yesterday's code.

Skip one step and see what your visitors get

YOU, at 5pm on a Thursday

commit + push

docker build

docker push

ssh in

docker pull

docker rm -f

docker run

elapsed 0s

cloud-explorer:latest

waiting on docker pull

YOUR SERVER

badge

the live badge

by hand

GitHub Actions

commit + push

docker build

docker push

ssh

docker pull

rm -f

docker run

reset

You changed **gradientColors** and want it live. Click the seven steps, in order.

click the seven steps in order – miss one and see what ships

Six secrets, so none of it ever sits in your code

GitHub Actions is the robot

It runs steps you define whenever something happens – here, a push to main. The steps are the same commands you just ran by hand.

Secrets, not source

Your private key and registry token go in Settings → Secrets and variables → Actions, never in the repository. The workflow reads them as `${{ secrets.NAME }}`.

DOCKERHUB_USERNAME	your Docker Hub username
DOCKERHUB_TOKEN	Docker Hub → Account Settings → Security → New Access Token
SSH_HOST	your server's public IP (the EIP)
SSH_USER	root, or your login user
SSH_KEY	the contents of your private .pem file
SITE_ADDRESS	your <ip-with-dashes>.sslip.io

Five blanks turn seven manual steps into one git push

```
on:
  push:
    branches: [_____]          # which branch deploys?

jobs:
  build-and-push:
    steps:
      - uses: docker/login-action@v4
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets._____ } # which secret?
      - uses: docker/build-push-action@v7

  deploy:
    needs: _____          # wait for which job?
    steps:
      - uses: appleboy/ssh-action@v1
        with:
          script: |
            docker rm -f cloud-explorer || true
            docker run -d --pull _____ -p 80:80 -p 443:443 \
              -e SITE_ADDRESS=${ secrets._____ } \
              --name cloud-explorer .../cloud-explorer:latest
```



MILESTONE 4 • FROM "PUSH CODE" TO "IT'S LIVE", AUTOMATICALLY

Change a colour, push, and watch it deploy itself

- 1 Complete and commit both tasks – the `Dockerfile` and `.github/workflows/deploy.yml`.
- 2 Confirm the badge is live by hand at least once.
- 3 Add all [six repository secrets](#) – create the Docker Hub access token first, and paste it into GitHub before you close the tab.
- 4 Change `gradientColors`, flip `isOnCloud` to `true`, then commit and push from VS Code.
- 5 Watch both jobs go green in the [Actions](#) tab, then refresh your site. New colours, and you did nothing but push.

☐ Badge live over HTTPS · pushing to `main` redeploys it automatically, with a green check in the Actions tab.

You built a deployment pipeline, and it works without you

- ☐ Your badge is live over HTTPS on your own server, and a push to `main` rebuilds the image, pushes it, and restarts the container – with no hands.

This morning

Forked and cloned a real project, made it yours, and ran containers on your cloud server.

This afternoon

Packaged your own app into a small image, put it live over HTTPS, and automated the deploy.

Tomorrow

You opened ports 80 and 443 on faith. Day 3 explains what a port, a subnet and a security group actually are.

 Kahoot: Day 2 · Session 2 – Dockerfiles, multi-stage builds, images vs containers, and CI/CD.

IN PARTNERSHIP WITH

